

React Basics — Self-Study Guide & Assignment

For: the front-end team (you already know HTML, CSS, JavaScript/ES6+, and TypeScript). **Goal:** learn React fundamentals **on your own**, then build one small app that proves you understand them. **Time:** ~1 week of evenings, or a focused weekend.

What is React, in one line? A JavaScript library for building user interfaces out of reusable **components** — small functions that return HTML-like markup (JSX) and automatically re-render when their data (**state**) changes. Instead of manually doing `document.getElementById(...).innerHTML = ...` (like our vanilla mini-projects), you describe *what* the UI should look like for a given state, and React updates the page for you.

Part 1 — Environment setup (Windows)

1. Install Node.js

1. Go to <https://nodejs.org> → download the **LTS** button.
2. Run the installer, accept defaults, finish.
3. Open **PowerShell** (Start menu → type "PowerShell") and check: `powershell node -v npm -v` Both should print a version. If "not recognized", close and reopen PowerShell (or restart the PC).

2. Install VS Code + helpful extensions

- Editor: <https://code.visualstudio.com> (Windows installer).
- In VS Code → Extensions (`Ctrl+Shift+X`), install:
- **ES7+ React/Redux/React-Native snippets** (fast component boilerplate)
- **Prettier** (auto-formatting)
- **Simple React Snippets** (optional)

3. Create a React project with Vite (the modern, fast way)

In PowerShell, go to the folder where you keep projects, then:

```
npm create vite@latest my-first-react-app -- --template react
cd my-first-react-app
npm install
npm run dev
```

- When it finishes, it prints a local URL like **`http://localhost:5173`** — open it in your browser.
- Edit `src/App.jsx`, save, and the browser updates instantly (hot reload).
- Stop the server with `Ctrl+C` in the terminal.

Notes: the `-- --template react` gives a **JavaScript** project (best for learning React first). Once comfortable, try `--template react-ts` to combine it with the TypeScript you just learned. Don't use the old `create-react-app` — Vite is faster and current.

4. Project tour (what the files are)

```
my-first-react-app/
├─ index.html      ← single HTML page; has <div id="root">
├─ src/
│  └─ main.jsx     ← entry point: mounts <App/> into #root
│  └─ App.jsx      ← your main component (start here)
│     └─ App.css   ← styles
└─ package.json    ← scripts & dependencies
```

Part 2 — Topics to learn (in this order)

Work top to bottom. Each builds on the last. The **official docs at <https://react.dev/learn> are excellent and free** — follow "Quick Start" and "Describing the UI" / "Adding Interactivity".

#	Topic	What it is / why it matters
1	Components	The building block: a function that returns UI. Everything in React is components you compose together.
2	JSX	The HTML-like syntax inside components. Learn the differences: <code>className</code> (not <code>class</code>), <code>{ }</code> to embed JS, one root element, self-closing tags.
3	Props	How a parent passes data <i>into</i> a child component (like function arguments). One-way, read-only.
4	Rendering lists	Use <code>array.map()</code> to turn data into elements; each needs a unique <code>key</code> prop. (Same <code>.map</code> from your JS lessons.)
5	Conditional rendering	Show/hide UI with <code>&&</code> , ternary <code>? :</code> , or early returns (e.g. loading vs loaded).
6	State — <code>useState</code>	Data that changes over time and re-renders the UI when it does. The #1 React concept.
7	Handling events	<code>onClick</code> , <code>onChange</code> , <code>onSubmit</code> — pass a function; update state inside it.
8	Forms / controlled inputs	Bind an input's <code>value</code> to state and update on <code>onChange</code> .
9	<code>useEffect</code>	Run side effects (like fetching data) after render; the dependency array <code>[]</code> controls when.
10	Fetching data from an API	<code>useEffect</code> + <code>fetch</code> + <code>useState</code> for data/loading/error — this ties directly to your vanilla mini-projects.
11	Lifting state up (<i>light</i>)	When two components need the same data, move state to their common parent.

Mental model to hold onto: $UI = f(state)$. You don't touch the DOM. You change **state**, and React re-renders the components that depend on it.

Free resources

- <https://react.dev/learn> — official, best starting point (interactive).
- Scrimba "Learn React" (free course) — hands-on.
- freeCodeCamp React section — practice.
- YouTube: search "React JS crash course" (any recent 2024/2025 one).

Part 3 — The Assignment

Build **one** small React app in the Vite project. Do it in two stages so each concept is exercised.

Stage A — Warm-up: a "Team Cards" page (components + props + lists)

Build a page that shows a list of profile cards. - Create a `ProfileCard` component that takes **props**: `name`, `role`, `avatarEmoji`. - In `App`, keep an **array** of 3–4 people and render a `ProfileCard` for each with `.map()` (remember the `key`). - Style it with CSS (or Tailwind if you've set it up).

Proves: components, JSX, props, list rendering with keys.

Stage B — Main task: a live search app (state + events + effects + fetch)

Reuse an API you already have docs for (pick one): **TheMealDB** (recipes), **GitHub Users**, **Open Library** (books), or **REST Countries**.

Requirements: 1. A **search box** (controlled input bound to `useState`). 2. On submit (or button click), **fetch** matching results from the API. 3. Show a **loading** state while fetching, and an **error** / **"no results"** message when appropriate (conditional rendering). 4. Render the results as a **list** of cards (`.map` + `key`) — reuse a `Card` component with props. 5. Nothing hard-coded — every visible value comes from the API response.

Proves: `useState` , events, controlled inputs, `useEffect` /async fetch, conditional rendering, lists, props — the whole core.

Concept → feature checklist (tick these off)

- [] At least 2 components, one reused via `.map()`
- [] Data passed via **props**
- [] `useState` for the search text and the results
- [] An **event handler** (`onChange` / `onSubmit` / `onClick`)
- [] `useEffect` or an async handler that calls `fetch`
- [] **Loading** and **error/empty** states shown conditionally
- [] A list rendered with a unique `key`

Stretch goals (optional)

- Add a "favourites" list (lift state up; toggle with a button).
- Convert the project to **TypeScript** (`react-ts` template) and type the API response with an `interface` (exactly like `ts-lesson/05-typing-a-real-api.ts`).
- Style with **Tailwind**.
- Add a detail view when a card is clicked.

Part 4 — How to submit / what "done" looks like

- `npm run dev` starts with no errors; the app works in the browser.
- The concept checklist above is fully ticked.
- A short note (3–5 lines): which API you used, and where each React concept shows up in your code.
- Bonus: run `npm run build` — it should succeed (proves it's production-ready).

Quick reference — the smallest React component

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);          // state
  return (
    <button onClick={() => setCount(count + 1)}>    {/* event + JSX */}
      Clicked {count} times
    </button>
  );
}

export default Counter;
```

That one snippet has a **component**, **JSX**, **state** (`useState`), and an **event handler** — the four ideas everything else builds on.

Tips while learning

- `className` , not `class` in JSX (a classic first mistake).
- Wrap multiple elements in one parent (or a `<...>` fragment) — a component returns **one** root.
- **Never mutate state directly** — always use the setter (`setCount(...)`), and for arrays/objects make a new copy (your spread `...` from the JS lesson!).
- Read the red error overlay — React's errors are very descriptive.
- Keep components small; when one gets big, split it.